Apache CXF - JAX-WS Client & Test

In this section, we will learn how to write a client that uses our service.

Developing Client

In the server application we've created in **Apache CXF - Create JAX WS Server** chapter, **HelloWorld** is the interface that exposes our web service. The web service itself just provides a plain greeting message to the client. In this trivial application, we will expose our web service to the client by exposing directly the service interface and that is the **HelloWorld.class**.

Writing the client in a CXF application is as trivial as writing a server. Here is the complete code for the client –

```
//Client.java
package com.tutorialspoint.cxf.jaxws.helloworld;
import javax.xml.namespace.QName;

import jakarta.xml.ws.Service;
import jakarta.xml.ws.soap.SOAPBinding;

public final class Client {
   private static final QName SERVICE_NAME
   = new QName("http://helloworld.jaxws.cxf.tutorialspoint.com/",
   "HelloWorld");
   private static final QName PORT_NAME
   = new QName("http://helloworld.jaxws.cxf.tutorialspoint.com/",
   "HelloWorldPort");
   private Client() {
   }
   public static void main(String[] args) throws Exception {
      Service service = Service.create(SERVICE_NAME);
      System.out.println("service created");
      String endpointAddress = "http://localhost:9090/HelloServerPort";
```

```
        service.addPort(PORT_NAME, SOAPBinding.SOAP11HTTP_BINDING,
        endpointAddress);
        HelloWorld hw = service.getPort(HelloWorld.class);
        System.out.println(hw.greetings("World"));
    }
}
```

Here, we use the CXF supplied **Service** class to bind to the known service. We call the **create** method on the **Service** class to get an instance of the service. We set the known port by calling the **addPort** method on the **service** instance.

Now, we are ready to consume the service, which we do by first obtaining the service interface by calling the **getPort** method on the **service** instance. Finally, we call our **greetings** method to print the greetings message on the console.

Now, as you have learned the basics of CXF by using the Apache CXF-First approach, you will now learn how to use CXF with WSDL-First approach in our next chapter.

Update pom.xml to include client profile

pom.xml

```
<?xml version = "1.0" encoding = "UTF-8"?>
<project xmlns = "http://maven.apache.org/POM/4.0.0" xmlns:xsi =
"http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation =
"http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <groupId>com.tutorialspoint</groupId>
   <artifactId>cxf-pojo</artifactId>
   <version>1.0</version>
   <packaging>jar</packaging>
   <properties>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
      <maven.compiler.source>24</maven.compiler.source>
      <maven.compiler.target>24</maven.compiler.target>
      <cxf.rt.version>4.1.3</cxf.rt.version>
   </properties>
```

```
    <profiles>
      <profile>
       <id>server</id>
       <build>
          <defaultGoal>test</defaultGoal>
          <plugins>
             <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>1.6.0</version>
                <executions>
                   <execution>
                      <phase>test</phase>
                      <goals>
                         <goal>java</goal>
                      </goals>
                      <configuration>
                         <mainClass>
                            com.tutorialspoint.cxf.jaxws.helloworld.Server
                         </mainClass>
                      </configuration>
                   </execution>
                </executions>
             </plugin>
          </plugins>
       </build>
    </profile>

     <profile>
          <id>client</id>
          <build>
             <defaultGoal>test</defaultGoal>
             <plugins>
                <plugin>
                   <groupId>org.codehaus.mojo</groupId>
                   <artifactId>exec-maven-plugin</artifactId>
                   <executions>
                      <execution>
                         <phase>test</phase>
                         <goals>
```

```
                            <goal>java</goal>
                         </goals>
                         <configuration>
                            <mainClass>

com.tutorialspoint.cxf.jaxws.helloworld.Client
                            </mainClass>
                         </configuration>
                      </execution>
                   </executions>
                </plugin>
             </plugins>
          </build>
       </profile>
    </profiles>

    <dependencies>
<!-- https://mvnrepository.com/artifact/org.apache.cxf/cxf-rt-features-
logging -->
<dependency>
    <groupId>org.apache.cxf</groupId>
    <artifactId>cxf-rt-features-logging</artifactId>
    <version>${cxf.rt.version}</version>
    <scope>test</scope>
</dependency>
       <dependency>
          <groupId>org.apache.cxf</groupId>
          <artifactId>cxf-rt-frontend-simple</artifactId>
          <version>${cxf.rt.version}</version>
       </dependency>
       <dependency>
          <groupId>org.apache.cxf</groupId>
          <artifactId>cxf-rt-transports-http</artifactId>
          <version>${cxf.rt.version}</version>
       </dependency>
       <dependency>
          <groupId>org.apache.cxf</groupId>
          <artifactId>cxf-rt-frontend-jaxws</artifactId>
          <version>${cxf.rt.version}</version>
       </dependency>
```

```
        <!-- Jetty is needed if you're using the CXFServlet -->
        <dependency>
           <groupId>org.apache.cxf</groupId>
           <artifactId>cxf-rt-transports-http-jetty</artifactId>
           <version>${cxf.rt.version}</version>
        </dependency>
     </dependencies>
  </project>
```

Running Client

Make sure that the server is still running on your machine. In case, if it has timed out, restart the server with the following command −

```
mvn -Pserver
```

You will see the following message on the console −

```
INFO: Setting the server's publish address to be
http://localhost:9090/HelloServerPort
Server ready...
```

Now, before the server times out which we have set to 5 minutes, open another command line window and start the client with the following command −

```
mvn -Pclient
```

You will see a message similar to the following on the command line −

```
INFO: Creating Service
{http://helloworld.jaxws.cxf.tutorialspoint.com/}HelloWorld from class
com.tutorialspoint.cxf.jaxws.helloworld.HelloWorld
hi World
Hi world
```